# LeetCode Practice

## 10 Medium: Sorting

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

**10**
**MEDIUM**

**Patterns (in order)**

1. Sorting (10)

**How to use**

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**10 questions | tap a title to jump to it**


| Medium questions (10) | Go > |
|---|---|
| **Sorting (10)** | **Go >** |
| M 1. Group Anagrams | LC 49 |
| M 2. Merge Intervals | LC 56 |
| M 3. Maximum Gap | LC 164 |
| M 4. Majority Element II | LC 229 |
| M 5. Queue Reconstruction by Height | LC 406 |
| M 6. Minimum Area Rectangle | LC 939 |
| M 7. Reveal Cards In Increasing Order | LC 950 |
| M 8. Analyze User Website Visit Pattern | LC 1152 |
| M 9. Remove Covered Intervals | LC 1288 |
| M 10. Determine if Two Strings Are Close | LC 1657 |

## Medium questions (10)

MEDIUM | TOPIC 1 OF 1

# Sorting

**10 Medium questions start on the next page**

**Before this:** this is the first topic
**After this:** this is the last topic

---

**IN THIS TOPIC**

**1.** Group Anagrams LC 49
**2.** Merge Intervals LC 56
**3.** Maximum Gap LC 164
**4.** Majority Element II LC 229
**5.** Queue Reconstruction by Height LC 406
**6.** Minimum Area Rectangle LC 939
**7.** Reveal Cards In Increasing Order LC 950
**8.** Analyze User Website Visit Pattern LC 1152
**9.** Remove Covered Intervals LC 1288
**10.** Determine if Two Strings Are Close LC 1657

MEDIUM | Sorting | LeetCode #49 | 1 of 10 | 1/10

# Group Anagrams

## QUESTION

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

**Example 1**

| | |
|---|---|
| Input | `strs = ["eat","tea","tan","ate","nat","bat"]` |
| Output | `[["bat"],["nat","tan"],["ate","eat","tea"]]` |

**Example 2**

| | |
|---|---|
| Input | `strs = [""]` |
| Output | `[[""]]` |

**Example 3**

| | |
|---|---|
| Input | `strs = ["a"]` |
| Output | `[["a"]]` |

CONSTRAINTS

- 1 <= strs.length <= $10^4$
- 0 <= strs[i].length <= 100
- strs[i] consists of lowercase English letters.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Group strings that are anagrams. Return groups in any order.

Same chars, same frequencies -> anagrams. Empty string is its own group. Right?"

"['eat','tea','tan','ate','nat','bat'] -> 3 groups ok"

Are string comparisons case-sensitive?

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
groups = []
for word in words:
    compare its sorted letters with those of the
        first word of every group
    add it to the group that matches, else start
        a new group
```

| TIME | SPACE |
|---|---|
| O(n * g * L log L) | O(n) |

WHY IT IS SLOW

### PHASE 2 BRUTE FORCE

Each word is compared with many groups, O(n * groups * L log L). Using the sorted letters (or a 26-count tuple) as a hash-map key groups in one pass.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Sort each word as key, group by key — O(n * k log k). Walk-through:"*

**For each word in strs:**

`key = ''.join(sorted(word)) <- anagrams share the same sorted key.`

groups[key].append(word).

Return list(groups.values()).

**WHY sorted key:** anagrams are permutations of each other -> sorting them gives identical strings.

'eat','tea','ate' all sort to 'aet' -> same group.

Alternative: count-tuple key (26-char frequency) -> O(n*k) but more code.

**Time:** O(n * k log k) where k = max word length. Space: O(n*k).

### PHASE 4 CLEAN CODE

```
class _49_GroupAnagrams:
    def groupAnagrams(self, strs: List[str]) -> List[List[
str]]:
        groups: defaultdict = defaultdict(list)
        for word in strs:
            groups[tuple(sorted(word))].append(word)
        return list(groups.values())
```

### PHASE 5 TEST & DEBUG

*"Let me trace groupAnagrams(['eat','tea','tan','ate','nat','bat'])."*

Key each word by sorted characters.

'eat'->'aet'. 'tea'->'aet'. 'tan'->'ant'. 'ate'->'aet'. 'nat'->'ant'. 'bat'->'abt'.

Groups: {'aet':['eat','tea','ate'], 'ant':['tan','nat'], 'abt':['bat']}.

Return [['eat','tea','ate'],['tan','nat'],['bat']] ok

**WHY sorted key:** anagrams have identical character multisets -> same sorted string.

O(nklogk) where k=max word length (sort each word).

**WHY sorted string as key:** anagrams are permutations -> same character multisets -> sort normalizes.

Counter key alt: tuple(sorted(Counter(word).items())) works too but slower than sorted string.

Trace: ["eat","tea","tan","ate","nat","bat"] -> sort each: "aet","aet","ant","aet","ant","abt" -> groups: {"aet":["eat","tea","ate"],"ant":["tan","nat"],"abt":["bat"]}.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| O(n * m log m) | O(n * m) |

Frequency tuple alternative: O(n * m), avoids log m sort. For short words (typical)

sorting is fine; for very long words, frequency tuple wins.

**Given more time I'd ask:** Unicode input? sorted() handles it; frequency array breaks.

Use Counter-based key: tuple(sorted(Counter(word).items())).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sorting | LeetCode #56 | 2 of 10 | 2/10

# Merge Intervals

## QUESTION

Given an array of intervals where intervals[i] = [start$_i$, end$_i$], merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

| Example 1 | |
|---|---|
| Input | `intervals = [[1,3],[2,6],[8,10],[15,18]]` |
| Output | `[[1,6],[8,10],[15,18]]` |
| Explanation | *Since intervals [1,3] and [2,6] overlap, merge them into [1,6].* |

| Example 2 | |
|---|---|
| Input | `intervals = [[1,4],[4,5]]` |
| Output | `[[1,5]]` |
| Explanation | *Intervals [1,4] and [4,5] are considered overlapping.* |

| Example 3 | |
|---|---|
| Input | `intervals = [[4,7],[1,4]]` |
| Output | `[[1,7]]` |
| Explanation | *Intervals [1,4] and [4,7] are considered overlapping.* |

CONSTRAINTS

- 1 <= intervals.length <= $10^4$
- intervals[i].length == 2
- 0 <= start$_i$ <= end$_i$ <= $10^4$

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Merge all overlapping intervals. Two intervals overlap if one starts before or when the other ends.

[1,4] and [4,5] overlap at 4. Sort by start, then merge. Return non-overlapping result. Right?"

"[[1,3],[2,6],[8,10],[15,18]] -> [[1,6],[8,10],[15,18]] ok (1–3 and 2–6 overlap at 2-3)"

Are there duplicate elements? Should the sort be stable?

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
while True:
    for (a, b) in pairs(intervals):
        if they overlap:
            replace both by their union
    if no pair overlaps:
        break
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n) |

WHY IT IS SLOW

Each pass is O(n^2) and there can be n passes. Sort by start, then sweep once, extending the last merged interval while the next one overlaps: O(n log n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

Sort by start time — O(n log n). Then one pass:

Keep a result list. For each interval, if it overlaps with the last result interval

(last.end >= current.start), extend the last interval's end.

Otherwise, append as a new separate interval.

**Time:** O(n log n). Space: O(n)."

Key insight: sorting by start time guarantees that intervals that can merge are adjacent — one linear pass then extends or appends, making the merge O(n) after the O(n log n) sort.

### PHASE 4 CLEAN CODE

```
class _56_MergeIntervals:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        merged = [intervals[0]]
        for start, end in intervals[1:]:
            if merged[-1][1] >= start:                # overlap
                merged[-1][1] = max(merged[-1][1], end)
            else:
                merged.append([start, end])
        return merged
```

### PHASE 5 TEST & DEBUG

*"Let me trace merge([[1,3],[2,6],[8,10],[15,18]])."*

Sort by start (already sorted). result=[[1,3]].

[2,6]: 2<=3 -> overlap. Merge: [1,max(3,6)]=[1,6]. result=[[1,6]].

[8,10]: 8>6 -> no overlap. Append. result=[[1,6],[8,10]].

[15,18]: 15>10 -> no overlap. Append. result=[[1,6],[8,10],[15,18]] ok

[[1,4],[4,5]]: 4<=4 -> overlap -> [1,5] ok

**WHY sort first:** ensures we only need to compare with the last merged interval.

If current start > last end: new group. Else: extend last end to max of both ends.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| O(n log n) | O(n) |

The sort is the bottleneck — the merge pass itself is O(n).

If the input were already sorted, this runs in O(n) total.

**Given more time I'd ask:** how many meeting rooms needed?

That's Meeting Rooms II (#253) — use a min-heap of end times.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sorting | LeetCode #164 | 3 of 10 | 3/10

# Maximum Gap

## QUESTION

Given an integer array nums, return the maximum difference between two successive elements in its sorted form. If the array contains less than two elements, return 0.

You must write an algorithm that runs in linear time and uses linear extra space.

| Example 1 | |
|---|---|
| Input | `nums = [3,6,9,1]` |
| Output | `3` |
| Explanation | *The sorted form of the array is [1,3,6,9], either (3,6) or (6,9) has the maximum difference 3.* |

| Example 2 | |
|---|---|
| Input | `nums = [10]` |
| Output | `0` |
| Explanation | *The array contains less than 2 elements, therefore return 0.* |

CONSTRAINTS

- 1 <= nums.length <= $10^5$
- 0 <= nums[i] <= $10^9$

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the maximum gap between successive elements in the sorted form of the array.

Must run in linear time and space. Right?"

"A few quick questions:

If length < 2, return 0? Yes. All elements equal -> 0? Yes."

"Let me walk: nums=[3,6,9,1] -> sorted=[1,3,6,9] -> gaps=[2,3,3] -> max=3 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
sort a copy of nums
return the largest difference between neighbours
```

| TIME | SPACE |
|---|---|
| O(n log n) | O(n) |

WHY IT IS SLOW

### PHASE 2 BRUTE FORCE

The sort is the cost. Pigeonhole buckets (size about (max - min) / (n - 1), keep each bucket's min and max) give the answer in O(n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is O(n log n) sort.

Bucket sort (radix sort): place n numbers in [min,max] into n-1 buckets.

By pigeonhole, max gap >= (max-min)/(n-1), so the max gap must cross bucket boundaries.

Track min and max of each bucket; max gap = max difference between consecutive non-empty buckets.

**Time:** O(n). Space: O(n).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _164_MaximumGap:
    def maximumGap(self, nums):
        n = len(nums)
        if n < 2:
            return 0
        lo, hi = min(nums), max(nums)
        if lo == hi:
            return 0
        bucket_size = max(1, (hi - lo) // (n - 1))
        bucket_count = (hi - lo) // bucket_size + 1
        buckets = [[float('inf'), float('-inf')] for _ in
range(bucket_count)]
        for num in nums:
            idx = (num - lo) // bucket_size
            buckets[idx][0] = min(buckets[idx][0], num)
            buckets[idx][1] = max(buckets[idx][1], num)
        max_gap = 0
        prev_max = lo
        for b_min, b_max in buckets:
            if b_min == float('inf'):
                continue
            max_gap = max(max_gap, b_min - prev_max)
            prev_max = b_max
        return max_gap
```

### PHASE 5 TEST & DEBUG

*"Let me trace after sorting—what property does sorting give me, and how the scan uses it."*

nums=[3,6,9,1]: lo=1,hi=9,bucket_size=max(1,(9-1)//3)=2. 4 buckets.

3->bucket1[1,3], 6->bucket2[5,7]? No: (3-1)//2=1->bucket1. (6-1)//2=2->bucket2. (9-1)//2=4->bucket4. (1-1)//2=0->bucket0.

bucket0=[1,1], bucket1=[3,3], bucket2=[6,6], bucket3=[inf,-inf], bucket4=[9,9].

Gaps: 3-1=2, 6-3=3, 9-6=3. max=3 ok

Bucket size ensures max gap > bucket size, so gaps only occur between buckets

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n)** | **O(n)** for buckets |

**Follow-up:** Sort Colors (#75) — bucket sort for 3 values.

**Follow-up:** if elements are large but sparse, use radix sort on digits.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sorting | LeetCode #229 | 4 of 10 | 4/10

# Majority Element II

## QUESTION

Given an integer array of size n, find all elements that appear more than n/3 times.

| Example 1 | |
|---|---|
| Input | `nums = [3,2,3]` |
| Output | `[3]` |

| Example 2 | |
|---|---|
| Input | `nums = [1]` |
| Output | `[1]` |

| Example 3 | |
|---|---|
| Input | `nums = [1,2]` |
| Output | `[1,2]` |

CONSTRAINTS

- 1 <= nums.length <= $5 * 10^4$
- $10^9$ <= nums[i] <= $10^9$
- Follow up: Could you solve the problem in linear time and in O(1) space?

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all elements that appear MORE THAN n/3 times.

At most 2 such elements can exist (since 3*(n/3+1) > n). Right?"

"A few quick questions:

Return in any order? Yes. If none qualify, return []? Yes."

"Let me walk: nums=[3,2,3] -> 3 appears 2>1 times -> [3] ok. nums=[1,2] -> [] ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for element in elements:
    count its occurrences by scanning the array
    keep those occurring more than n/3 times
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

### PHASE 2 BRUTE FORCE

A scan per distinct value. Boyer-Moore voting with two candidates and a verification pass is O(n) time and O(1) space (at most two values exceed n/3).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is the O(n) extra Counter space.

Boyer-Moore Voting for n/3 threshold: maintain TWO candidates and TWO counts.

When adding a vote: if it matches a candidate, increment; else decrement both;

**`if a count is 0, replace that candidate.`**

Second pass: verify both candidates actually exceed n/3.

**Time:** O(n). Space: O(1)."

Key insight: Boyer-Moore Voting extends to n/3 by maintaining two candidate-count pairs — any element appearing > n/3 times must survive the cancellation phase.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _229_MajorityElementII:
    def majorityElement(self, nums):
        cand1 = cand2 = None
        count1 = count2 = 0
        for num in nums:
            if num == cand1: count1 += 1
            elif num == cand2: count2 += 1
            elif count1 == 0: cand1, count1 = num, 1
            elif count2 == 0: cand2, count2 = num, 1
            else: count1 -= 1; count2 -= 1
        threshold = len(nums) // 3
        return [c for c in [cand1, cand2] if c is not None
 and nums.count(c) > threshold]
```

### PHASE 5 TEST & DEBUG

*"Let me trace after sorting—what property does sorting give me, and how the scan uses it."*

nums=[3,2,3]: cand1=3,c1=1. cand2=2,c2=1. 3==cand1->c1=2. cands=[3,2].

verify: 3.count=2>1 ok, 2.count=1>1 . -> [3] ok

nums=[1,1,1,3,3,2,2,2]: cands end as [1,2] or [2,1]. Both appear 3 times > 8/3=2. -> [1,2] ok

The verification pass is essential — candidates may not truly exceed n/3

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n)** | **O(1)** |
| two passes | only four variables |

Generalises Boyer-Moore: for n/k threshold, maintain k-1 candidates.

**Follow-up:** Majority Element I (#169) — n/2 threshold; one candidate.

**Follow-up:** count frequency of majority elements after finding them.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sorting | LeetCode #406 | 5 of 10 | 5/10

# Queue Reconstruction by Height

## QUESTION

You are given an array of people, people, which are the attributes of some people in a queue (not necessarily in order). Each people[i] = [$h_i$, $k_i$] represents the ith person of height $h_i$ with exactly $k_i$ other people in front who have a height greater than or equal to $h_i$.

Reconstruct and return the queue that is represented by the input array people. The returned queue should be formatted as an array queue, where queue[j] = [$h_j$, $k_j$] is the attributes of the jth person in the queue (queue[0] is the person at the front of the queue).

### Example 1

| | |
|---|---|
| Input | `people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]` |
| Output | `[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]` |
| Explanation | *Person 0 has height 5 with no other people taller or the same height in front.*<br>*Person 1 has height 7 with no other people taller or the same height in front.*<br>*Person 2 has height 5 with two persons taller or the same height in front, which is person 0 and 1.*<br>*Person 3 has height 6 with one person taller or the same height in front, which is person 1.*<br>*Person 4 has height 4 with four people taller or the same height in front, which are people 0, 1, 2, and 3.*<br>*Person 5 has height 7 with one person taller or the same height in front, which is person 1.*<br>*Hence [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] is the reconstructed queue.* |

### Example 2

| | |
|---|---|
| Input | `people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]` |
| Output | `[[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]` |

### CONSTRAINTS

- 1 <= people.length <= 2000
- 0 <= $h_i$ <= $10^6$
- 0 <= $k_i$ < people.length
- It is guaranteed that the queue can be reconstructed.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

People described as [height, k]: reconstruct queue where k = number of taller/equal people before them.

Right?"

"A few quick questions:

Sort first, then insert at position k? Yes."

"Let me walk: people=[[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]] -> [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(people):
    if every person has exactly k taller-or-equal
        people in front:
        return it
```

| TIME | SPACE |
|---|---|
| O(n! * n^2) | O(n) |

WHY IT IS SLOW

n! orderings. Sort by height descending (and k ascending for equal heights), then insert each person at index k: O(n^2) with a list, O(n log n) with a tree.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is O(n²) list insertions.

The sort + insert approach IS the standard greedy — O(n²) is acceptable.

With a BIT (Binary Indexed Tree) or segment tree: O(n log n) — complex to implement.

For interviews, the O(n²) solution is expected.

**Time:** O(n²). Space: O(n)."

Key insight: placing taller people first means shorter people inserted later never affect taller people's k-counts — the greedy sort makes each insertion position exactly correct.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _406_QueueReconstructionByHeight:
    def reconstructQueue(self, people):
        people.sort(key=lambda x: (-x[0], x[1]))
        result = []
        for person in people:
            result.insert(person[1], person)
        return result
```

## PHASE 5 TEST & DEBUG

*"Let me trace reconstructQueue([[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]])."*

Sort by height desc, then k asc: [[7,0],[7,1],[6,1],[5,0],[5,2],[4,4]].

**Insert each at position k:**

[7,0] -> insert at 0: [[7,0]].

[7,1] -> insert at 1: [[7,0],[7,1]].

[6,1] -> insert at 1: [[7,0],[6,1],[7,1]].

[5,0] -> insert at 0: [[5,0],[7,0],[6,1],[7,1]].

[5,2] -> insert at 2: [[5,0],[7,0],[5,2],[6,1],[7,1]].

[4,4] -> insert at 4: [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]].

Return [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] ok

**WHY sort tallest first:** shorter people don't affect count of taller people in front.

Sorted by height desc, k asc. Greedy insert at position k. deque=[]: insert (7,0)->[(7,0)]; insert (7,1)->[(7,0),(7,1)]; insert (6,1)->[(7,0),(6,1),(7,1)]. queue built by insertions ok

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n²)** n insertions each O(n) for list shifting | **O(n)** |

**Follow-up:** O(n log n) — use a Fenwick tree to find kth empty position.

**Follow-up:** if heights can be equal, the tiebreak by k ascending is critical.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sorting | LeetCode #939 | 6 of 10 | 6/10

# Minimum Area Rectangle

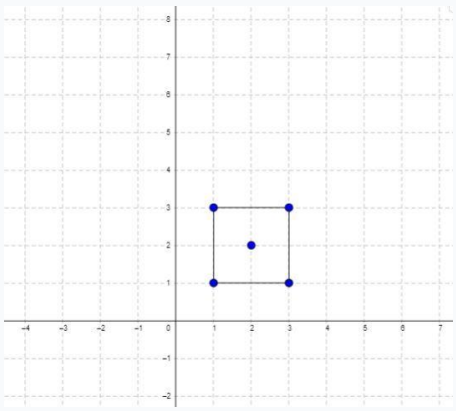
## QUESTION

You are given an array of points in the X-Y plane points where points[i] = [$x_i$, $y_i$].

Return the minimum area of a rectangle formed from these points, with sides parallel to the X and Y axes. If there is not any such rectangle, return 0.

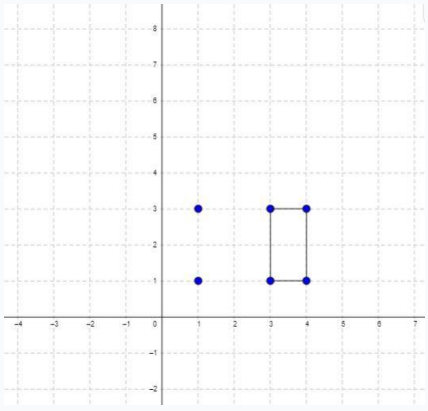
**Example 1**

| | |
|---|---|
| Input | `points = [[1,1],[1,3],[3,1],[3,3],[2,2]]` |
| Output | `4` |



**Example 2**

| | |
|---|---|
| Input | `points = [[1,1],[1,3],[3,1],[3,3],[4,1],[4,3]]` |
| Output | `2` |



CONSTRAINTS

- 1 <= points.length <= 500
- points[i].length == 2
- 0 <= $x_i$, $y_i$ <= 4 * $10^4$
- All the given points are unique.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given set of points, find the minimum area of an axis-aligned rectangle formed by 4 points. Return 0 if none. Right?"

"A few quick questions:

Axis-aligned means sides parallel to x and y axes? Yes."

"Let me walk: points=[[1,1],[1,3],[3,1],[3,3],[2,2]] -> min area=4 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for set in four_points:
    check whether they form an axis-aligned rectangle
    best = min(best, area)
```

| TIME | SPACE |
|---|---|
| O(n^4) | O(1) |

WHY IT IS SLOW

n^4/24 combinations. Put the points in a set; for every pair of points that could be opposite corners, check that the other two corners exist: O(n^2).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Fix two columns, find common y-coordinates — O(n²). Walk-through:"*

Group points by x-coordinate: col[x] = set of y values.

For each pair of columns (x1, x2):

Common y-values = col[x1] col[x2].

If >=2 common y-values: rectangle exists.

Min area = (x2-x1) * min distance between any two shared y-values.

Update global minimum.

**WHY sort y-values per column:** find minimum gap between shared y's efficiently.

**WHY fix columns:** a rectangle is determined by its two x-coordinates and two y-coordinates.

Iterating all O(n²) column pairs and their shared rows covers all rectangles.

**Time:** O(n²). Space: O(n).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _939_MinimumAreaRectangle:
    def minAreaRect(self, points):
        point_set = set(map(tuple, points))
```

```
        best = float('inf')
        n = len(points)
        for i in range(n):
            for j in range(i + 1, n):
                x1, y1 = points[i]
                x2, y2 = points[j]
                if x1 != x2 and y1 != y2:
                    if (x1, y2) in point_set and (x2, y1)
in point_set:
                        best = min(best, abs(x2 - x1) * ab
s(y2 - y1))
        return best if best != float('inf') else 0
```

## PHASE 5 TEST & DEBUG

*"Let me trace after sorting—what property does sorting give me, and how the scan uses it."*

points=[[1,1],[1,3],[3,1],[3,3],[2,2]]:

Pair (1,1),(3,3): (1,3) and (3,1) in set -> area=4. Pair (1,3),(3,1): same -> area=4. min=4 ok

X1!=x2 and y1!=y2 ensures a proper rectangle, not a degenerate line

Edge: fewer than 4 points -> no rectangle possible -> 0 ok. All collinear points -> no rectangle. Duplicate points handled by set membership.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n²)** | **O(n)** |

**Follow-up:** Minimum Area Rectangle II (#963) — any rotation allowed; harder.

**Follow-up:** Valid Square (#593) — check if 4 given points form a square.

MEDIUM | Sorting | LeetCode #950 | 7 of 10 | 7/10

# Reveal Cards In Increasing Order

## QUESTION

You are given an integer array deck. There is a deck of cards where every card has a unique integer. The integer on the ith card is deck[i].

You can order the deck in any order you want. Initially, all the cards start face down (unrevealed) in one deck.

You will do the following steps repeatedly until all cards are revealed:

### CONSTRAINTS

- 1. Take the top card of the deck, reveal it, and take it out of the deck.
- 2. If there are still cards in the deck then put the next top card of the deck at the bottom of the deck.
- 3. If there are still unrevealed cards, go back to step 1. Otherwise, stop.

Return an ordering of the deck that would reveal the cards in increasing order.

Note that the first entry in the answer is considered to be the top of the deck.

| Example 1 | |
|---|---|
| Input | `deck = [17,13,11,2,3,5,7]` |
| Output | `[2,13,3,11,5,17,7]` |
| Explanation | *We get the deck in the order [17,13,11,2,3,5,7] (this order does not matter), and reorder it.<br>After reordering, the deck starts as [2,13,3,11,5,17,7], where 2 is the top of the deck.<br>We reveal 2, and move 13 to the bottom. The deck is now [3,11,5,17,7,13].<br>We reveal 3, and move 11 to the bottom. The deck is now [5,17,7,13,11].<br>We reveal 5, and move 17 to the bottom. The deck is now [7,13,11,17].<br>We reveal 7, and move 13 to the bottom. The deck is now [11,17,13].<br>We reveal 11, and move 17 to the bottom. The deck is now [13,17].<br>We reveal 13, and move 17 to the bottom. The deck is now [17].<br>We reveal 17.<br>Since all the cards revealed are in increasing order, the answer is correct.* |

| Example 2 | |
|---|---|
| Input | `deck = [1,1000]` |
| Output | `[1,1000]` |

- 1 <= deck.length <= 1000
- 1 <= deck[i] <= $10^6$
- All the values of deck are unique.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Simulate: repeatedly take the top card and put it face up, then move the next card to the bottom.

Return the resulting deck order. Right?"

"A few quick questions:

Cards sorted in increasing order in the output? Yes — we work backwards to find initial order."

"Let me walk: deck=[17,13,11,2,3,5,7] -> [2,13,3,11,5,17,7] ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

#### BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(cards):
    simulate the reveal process (show the top,
        move the next to the bottom)
return the first ordering that reveals them in
    increasing order
```

| TIME | SPACE |
|---|---|
| O(n! * n) | O(n) |

#### WHY IT IS SLOW

n! orderings. Simulate the reveal on indices 0..n-1 with a queue to learn which index is revealed k-th, then place the k-th smallest card there: O(n log n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Simulate with a deque — O(n log n). Walk-through:"*

Sort nums. Simulate the 'reveal' process in REVERSE to find the initial arrangement.

Use a deque of indices 0..n-1.

**Work backwards from the last card to be revealed:**

Place largest remaining num at the front of the deque (at the index to be revealed last).

Rotate deque: move the last element to the front (reverse of 'move to bottom' operation).

OR: simulate forward directly:

`deque = [0,1,2,...,n-1] (positions). result = [0]*n.`

**For each num in sorted nums:**

result[deque.popleft()] = num. (reveal position)

`deque.append(deque.popleft()) (move top to bottom)`

**Time:** O(n log n) for sorting. Space: O(n)."

Key insight: simulating the deal in reverse (using the deque of sorted positions) places each card directly into its final revealed index — O(n) without physical simulation.

## PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import deque
class _950_RevealCardsInIncreasingOrder:
    def deckRevealedIncreasing(self, deck):
        n = len(deck)
        deck.sort()
        indices = deque(range(n))
        result = [0] * n
        for card in deck:
            result[indices.popleft()] = card
            if indices:
                indices.append(indices.popleft())
        return result
```

## PHASE 5 TEST & DEBUG

*"Let me trace after sorting—what property does sorting give me, and how the scan uses it."*

deck=[17,13,11,2,3,5,7] -> sorted=[2,3,5,7,11,13,17]. indices=deque([0,1,2,3,4,5,6]).

card=2: result[0]=2, rotate->indices=[2,3,4,5,6,1]. card=3: result[2]=3, rotate->[4,5,6,1,3].

card=5: result[4]=5, rotate->[6,1,3,5]. card=7: result[6]=7, rotate->[1,3,5].

card=11: result[1]=11, rotate->[3,5]. card=13: result[3]=13, rotate->[5]. card=17: result[5]=17.

-> [2,11,3,13,5,17,7] ok

The index queue simulates the reveal-and-move process on positions

deque=[0,1,2,3,4,5,6] (indices). Pop front->assign result[0]=cards[0]=2. Rotate: appendleft(deque.popleft()) skips one. deque=[2,3,4,5,6,1]. Next: result[2]=3. deque=[4,5,6,1,3] ok

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n log n)** sort + O(n) simulation | **O(n)** |

**Follow-up:** if we want minimum cards to reveal in sorted order — same simulation.

**Follow-up:** Dota2 Senate (#649) — similar circular queue elimination.

MEDIUM | Sorting | LeetCode #1152 | 8 of 10 | 8/10

# Analyze User Website Visit Pattern

## QUESTION

You are given two string arrays username and website and an integer array timestamp. All the given arrays are of the same length and the tuple [username[i], website[i], timestamp[i]] indicates that the user username[i] visited the website website[i] at time timestamp[i].

A pattern is a list of three websites (not necessarily distinct).

CONSTRAINTS

- For example, ["home", "away", "love"], ["leetcode", "love", "leetcode"], and ["luffy", "luffy", "luffy"] are all patterns.

The score of a pattern is the number of users that visited all the websites in the pattern in the same order they appeared in the pattern.

- For example, if the pattern is ["home", "away", "love"], the score is the number of users x such that x visited "home" then visited "away" and visited "love" after that.
- Similarly, if the pattern is ["leetcode", "love", "leetcode"], the score is the number of users x such that x visited "leetcode" then visited "love" and visited "leetcode" one more time after that.
- Also, if the pattern is ["luffy", "luffy", "luffy"], the score is the number of users x such that x visited "luffy" three different times at different timestamps.

Return the pattern with the largest score. If there is more than one pattern with the same largest score, return the lexicographically smallest such pattern.

Note that the websites in a pattern do not need to be visited contiguously, they only need to be visited in the order they appeared in the pattern.

## QUESTION

### Example 1

| | |
|---|---|
| Input | `username = ["joe","joe","joe","james","james","james","james","mary","mary","mary"], timestamp = [1,2,3,4,5,6,7,8,9,10], website = ["home","about","career","home","cart","maps","home","home","about","career"]` |
| Output | `["home","about","career"]` |
| Explanation | *The tuples in this example are: ["joe","home",1],["joe","about",2],["joe","career",3],["james","home",4],["james","cart",5],["james","maps",6],["james","home",7],["mary","home",8],["mary","about",9], and ["mary","career",10].*<br>*The pattern ("home", "about", "career") has score 2 (joe and mary).*<br>*The pattern ("home", "cart", "maps") has score 1 (james).*<br>*The pattern ("home", "cart", "home") has score 1 (james).*<br>*The pattern ("home", "maps", "home") has score 1 (james).*<br>*The pattern ("cart", "maps", "home") has score 1 (james).*<br>*The pattern ("home", "home", "home") has score 0 (no user visited home 3 times).* |

### Example 2

| | |
|---|---|
| Input | `username = ["ua","ua","ua","ub","ub","ub"], timestamp = [1,2,3,4,5,6], website = ["a","b","a","a","b","c"]` |
| Output | `["a","b","a"]` |

- 3 <= username.length <= 50
- 1 <= username[i].length <= 10
- timestamp.length == username.length
- 1 <= timestamp[i] <= $10^9$
- website.length == username.length
- 1 <= website[i].length <= 10
- username[i] and website[i] consist of lowercase English letters.
- It is guaranteed that there is at least one user who visited at least three websites.
- All the tuples [username[i], timestamp[i], website[i]] are unique.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Find the 3-website pattern (subsequence, not necessarily consecutive) that the most DISTINCT users

have visited in order. On tie, return lexicographically smallest. Right?

Each user contributes at most 1 count per pattern regardless of how many times they visit."

## PHASE 1 CLARIFY

"joe visited home->about->career, james visited home->cart->maps->home, mary visited home->about->career.

Pattern ('home','about','career') score=2 (joe, mary) -> winner ok"

Are there duplicate elements? Should the sort be stable?

## PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for pattern in patterns:
    # of three websites (W^3)
    count the users who visited them in that order
return the pattern with the highest count
    (alphabetically first on ties)
```

| TIME | SPACE |
|---|---|
| O(W^3 * n) | O(W^3) |

WHY IT IS SLOW

Tries patterns that nobody visited. Sort each user's visits by time, generate that user's distinct 3-site patterns directly (at most C(v, 3)), and count each pattern's users in a hash map.

*"Should I go ahead and optimize?"*

## PHASE 3 OPTIMIZE

Sort all visits by timestamp. Group by user into ordered website lists.

For each user, generate all C(len,3) 3-subsequence combos — at most C(50,3)=19600.

Use a set per user to avoid double-counting (one user = one score per pattern).

Count across users with a global Counter.

Return pattern with max score, lexicographically smallest on ties."

**Time:** O(n). Space: O(n).

## PHASE 4 CLEAN CODE

```
class _1152_AnalyzeWebsiteVisit:
    def mostVisitedPattern(self, username: List[str], time
stamp: List[int], website: List[str]) -> List[str]:
        visits = sorted(zip(timestamp, username, website))
        user_sites: defaultdict = defaultdict(list)
        for _, user, site in visits:
            user_sites[user].append(site)

        pattern_score: Counter = Counter()
        for sites in user_sites.values():
            seen = set()
            for combo in itertools.combinations(sites, 3):
                if combo not in seen:
                    seen.add(combo)
                    pattern_score[combo] += 1

        return list(max(pattern_score, key=lambda p: (patt
ern_score[p], [-ord(c) for c in ''.join(p)])))
```

## PHASE 5 TEST & DEBUG

*"Let me trace mostVisitedPattern([[u1,u2,u3],[u1,u2,u3],[u1,u2]], [1,2,3],[1,2,3],[4,5,6])."*

For each user, collect sorted(set of 3-tuples from their sequence)

user1 visits 1,2,3 (twice): 3-sequences: (1,2,3)

user2 visits 1,2,3: 3-sequences: (1,2,3)

user3 visits 4,5,6: 3-sequences: (4,5,6)

Count unique users per 3-pattern: (1,2,3)->{u1,u2}=2, (4,5,6)->{u3}=1

Return (1,2,3) as list [1,2,3] ok

**WHY per-user dedup:** pattern that appears multiple times for same user counts once.

## PHASE 6 COMPLEXITY & FOLLOW-UP

Time O(n log n + U * C(K,3)) where U=users, K=max visits per user.

With U<=50, K<=50: C(50,3)=19600 * 50 = 980000 — manageable.

The per-user 'seen' set ensures one user contributes at most 1 to each pattern's score.

Tie-breaking: max() with a compound key — score ascending, then lexicographic ascending.

Given more time I'd ask: what if we want top-k patterns?

Use a heap instead of max() — same O complexity."

**Space:** O(log n) — sort stack; or O(1) for in-place heapsort variant.

MEDIUM | Sorting | LeetCode #1288 | 9 of 10 | 9/10

# Remove Covered Intervals

## QUESTION

Given an array intervals where intervals[i] = [$l_i$, $r_i$] represent the interval [$l_i$, $r_i$), remove all intervals that are covered by another interval in the list.

The interval [a, b) is covered by the interval [c, d) if and only if c <= a and b <= d.

Return the number of remaining intervals.

| Example 1 | |
|---|---|
| Input | `intervals = [[1,4],[3,6],[2,8]]` |
| Output | `2` |
| Explanation | *Interval [3,6] is covered by [2,8], therefore it is removed.* |

| Example 2 | |
|---|---|
| Input | `intervals = [[1,4],[2,3]]` |
| Output | `1` |

CONSTRAINTS

- 1 <= intervals.length <= 1000
- intervals[i].length == 2
- 0 <= $l_i$ < $r_i$ <= $10^5$
- All the given intervals are unique.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove all intervals [a,b] that are completely covered by another interval [c,d] (c<=a and b<=d).

Return count of remaining. Right?"

"A few quick questions:

An interval isn't removed if it's not completely covered by a DIFFERENT interval? Correct."

"Let me walk: intervals=[[1,4],[3,6],[2,8]] -> [2,8] covers [1,4]? No: 2>1. [2,8] covers [3,6]? 2<=3 and 6<=8 ok. Remove [3,6]. Remaining: [[1,4],[2,8]] -> 2 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for interval in intervals:
    compare it with every other interval
    count those not covered by another
```

### PHASE 2 BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(n^2) | O(1) |

WHY IT IS SLOW

Compares every pair. Sort by start ascending (and by end descending on ties) and sweep keeping the furthest end: an interval ending at or before it is covered: O(n log n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Sort by end time, skip if covered — O(n log n). Walk-through:"*

Sort intervals by start, then by end descending (so wider intervals come first at same start).

`max_right = 0. count = 0.`

For each [l, r] sorted by l ascending, r descending:

If r > max_right: max_right = r; count++.

Else: this interval is covered by a previous interval (its right <= current max_right and l is >= some prev l).

Return n - count. (removed = covered; count = uncovered; answer = n - uncovered... wait)

Actually count covered = n - uncovered.

Alternative: sort by start asc, end desc. Track max_end. If curr end <= max_end: covered (skip).

Else: max_end = curr end; not covered.

**Time:** O(n log n). Space: O(1).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1288_RemoveCoveredIntervals:
    def removeCoveredIntervals(self, intervals):
        intervals.sort(key=lambda x: (x[0], -x[1]))
        count = 0
        max_end = 0
        for start, end in intervals:
            if end > max_end:
                count += 1
                max_end = end
        return count
```

### PHASE 5 TEST & DEBUG

*"Let me trace removeCoveredIntervals([[1,4],[3,6],[2,8]])."*

Sort by start asc, then end desc (for same start, longer interval first).

Sorted: [[1,4],[1,8]? No: [1,4],[2,8],[3,6].

Actually sort: start asc, end desc for ties. [[1,4],[2,8],[3,6]].

Track max_right. [1,4]: max_right=4, count=3.

[2,8]: 8>4->not covered. max_right=8. count=3.

[3,6]: 6<=8->covered. count-=1=2.

Return 2 ok

**PHASE 6 COMPLEXITY & FOLLOW-UP**

| TIME | SPACE |
| --- | --- |
| O(n log n) | O(1) |

**Follow-up:** Merge Intervals (#56) — merge overlapping instead of removing covered.

**Follow-up:** Non-overlapping Intervals (#435) — remove minimum to make non-overlapping.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sorting | LeetCode #1657 | 10 of 10 | 10/10

# Determine if Two Strings Are Close

## QUESTION

Two strings are considered close if you can attain one from the other using the following operations:

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two strings are close if you can make one equal to the other using:

Op1: swap any two existing characters. Op2: transform all occurrences of one char to another.

They must have the same set of unique characters AND the same multiset of frequencies. Right?"

"A few quick questions:

Same length required? Not explicitly — but if char sets differ, return false.

Op2 maps ALL occurrences of one char to another existing char? Yes."

"Let me walk: w1='abc', w2='bca' -> same chars, same freqs [1,1,1] -> true ok

w1='aacabb', w2='bbcbaa' -> same chars {a,b,c}, freqs sorted [1,2,3]==[1,2,3] -> true ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
from word1, apply every possible swap of two letters
    and every swap of two letters' identities,
    breadth-first
return True if word2 appears
```

| TIME | SPACE |
|---|---|
| exponential | exponential |

WHY IT IS SLOW

Searches the space of strings. The strings are close exactly when they use the same set of letters and their sorted lists of letter counts are equal: O(n + 26 log 26).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is the combinatorial search — but the condition is simple:

1. Same set of unique characters (otherwise op2 can't bridge the gap).

### PHASE 3 OPTIMIZE

2. Same multiset of frequencies (op2 permutes freq values, op1 rearranges positions).

Both checks in O(n) time.

**Time:** O(n). Space: O(1) — at most 26 characters."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1657_DetermineIfTwoStringsClose:
    def closeStrings(self, word1, word2):
        from collections import Counter
        c1, c2 = Counter(word1), Counter(word2)
        return set(c1) == set(c2) and sorted(c1.values())
== sorted(c2.values())
```

### PHASE 5 TEST & DEBUG

*"Let me trace after sorting—what property does sorting give me, and how the scan uses it."*

w1='cabbba', w2='abbccc': c1={c:1,a:2,b:3}, c2={a:1,b:2,c:3}.

```
set(c1)={a,b,c}==set(c2) ok. sorted vals:
  [1,2,3]==[1,2,3] ok -> True ok
```

w1='a', w2='aa': set(c1)={a}==set(c2)={a} ok. sorted: [1]!=[2] -> False ok

Two conditions capture all valid transformations exactly

### PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n)** Counter build + O(26 log 26) sort ~ O(n) | **O(1)** 26 chars |

**Follow-up:** if Op2 could add new characters (not just rearrange existing), only freq multiset matters.

**Follow-up:** Minimum Number of Steps to Make Two Strings Anagram (#1347) — count missing chars.

IN YOUR APP | Expose | Editorial | Community